

Predicates, Not Assumptions:

A Soundness and Matched-Baseline Audit of LLM-Guided IC3IA

Pono-LLM Research Project
<https://github.com/swear01/pono-llm>
Archival Technical Report

Evidence frozen 14 July 2026; report packaged 18 July 2026

Abstract

Large language models can propose semantically plausible invariants, but the formal meaning of how those formulas enter a verifier is decisive. We report a soundness and matched-baseline audit of an LLM-guided extension of IC3IA in the PONO SMT-based model checker. The original system inserted LLM-proposed mutex formulas as BTOR2 constraints. This under-approximated the concrete transition system: the model checker proved a model in which behaviors had been assumed away. Independent certification on the original models rejected all 32 checkable legacy proofs, and bounded reachability found violations of 30 of 32 tested mutex formulas.

We replace assumption injection with initial *predicate* injection. Candidates enrich the implicit-abstraction vocabulary without changing the concrete initial states, transition relation, constraints, or safety properties. A false candidate can increase proof cost, but cannot manufacture a safety result by excluding concrete behaviors. We then re-evaluate the apparent gains using original-model C1/C2/C3 certification, frozen candidate replay, end-to-end cost accounting, and deterministic affine and quadratic candidate generators of matched expressiveness. On the corrected 21-task set, LLM and deterministic quadratic candidates certify exactly the same seven safe tasks; both corresponding portfolios reach eight safe and two unsafe tasks. A broader 1,919-file HWMCC census and a preregistered paired source/lifted/raw representation gate likewise leave no defensible LLM-specific solved-set or search-efficiency advantage.

The result is not a coverage-improvement claim. It is an empirical falsification and evaluation-methodology result: apparent LLM gains can arise from an unsound trust boundary, an expressiveness-mismatched baseline, or best-run accounting. The repository releases the sound integration, independent checkers, frozen artifacts, machine-readable claim ledger, and explicit not-run decisions.

1 Introduction

IC3/PDR proves safety by constructing an inductive strengthening rather than explicitly unrolling the transition relation [1]. IC3IA extends this idea to richer theories through implicit predicate abstraction [3], and PONO provides a flexible SMT-based implementation and research platform [5]. These techniques still depend critically on finding useful predicates and generalizations. An LLM appears attractive here: source-like state names and recurrence patterns can survive compilation into word-level BTOR2, while a language model can propose relations such as counter bounds, accumulator equalities, or polynomial recurrences.

The central danger is that a plausible formula is not evidence that the formula is true. A verifier integration must therefore answer two separate questions:

1. How may an untrusted proposal guide proof search without changing the concrete verification problem?
2. After sound integration, what improvement is attributable to the LLM rather than to the candidate language or additional search budget?

This project began with apparently strong positive results and ended with a negative marginal-value result. The reversal is scientifically useful because each stage identifies a concrete evaluation failure mode. First, LLM mutex hints had been encoded as assumptions, making the reported safety results unsound for the original models. Second, after replacing assumptions with sound abstraction predicates, the remaining “LLM-only” examples were only unique relative to verifier engines, not relative to deterministic generators of the same affine and quadratic forms. Third, broader and preregistered gates did not recover a source-representation, phase, grammar-routing, or compactness advantage.

The contributions of this report are:

- an original-model audit that rejects 32/32 checkable legacy proofs and directly demonstrates the danger of converting semantic hints into concrete assumptions;
- a sound initial-predicate interface for IC3IA in which arbitrary candidates refine the abstraction vocabulary but do not restrict the concrete model;
- independent bit-vector C1/C2/C3 checkers, every-property handling, exact Houdini filtering, frozen replay, and explicit UNKNOWN semantics;
- a matched-baseline evaluation showing that deterministic affine and quadratic candidate generators cover every current LLM solve;
- a sequence of broader, preregistered population and mechanism gates that records failures and *not-run* outcomes without moving the success criteria after observing results.

Our scoped conclusion is deliberately narrower than “LLMs do not help formal verification.” Under the models, candidate languages, budgets, and solver configurations evaluated here, no defensible LLM-specific solved-set or search-efficiency advantage survives soundness repair and matched deterministic comparison.

2 Background and Verification Semantics

2.1 Transition systems and safety

We model a word-level transition system as

$$M = (X, U, I(X), T(X, U, X'), C(X, U), \{B_j(X)\}_{j=1}^m),$$

where X are state variables, U are inputs, I is the initial-state formula, T is the transition relation, C contains original model constraints, and each B_j is a bad-state property. The evaluated tasks are primarily software-origin bit-vector models in BTOR2, a typed word-level transition-system format [6], drawn from Hardware Model Checking Competition distributions [4].

For a proposed invariant conjunction $H(X)$, an independent certificate requires, under exact fixed-width bit-vector semantics,

$$\text{C1: } I(X) \wedge C(X, U) \wedge \neg H(X) \quad \text{is UNSAT,} \quad (1)$$

$$\text{C2: } H(X) \wedge C \wedge T \wedge C' \wedge \neg H(X') \quad \text{is UNSAT,} \quad (2)$$

$$\text{C3}_j: H(X) \wedge C(X, U) \wedge B_j(X) \quad \text{is UNSAT} \quad (3)$$

for every bad property j . A satisfying query is a concrete refutation. An SMT timeout or UNKNOWN result is no evidence of induction.

2.2 IC3IA predicate abstraction

IC3IA performs IC3 over Boolean labels corresponding to theory predicates and incrementally refines an implicit abstraction. For a concrete predicate $p(X)$, the abstraction associates a Boolean label with $p(X)$ and a next-state label with $p(X')$. Adding a predicate partitions the concrete state space more finely and strengthens the abstract relational transition description, but a correct implicit abstraction remains an over-approximation: every concrete transition has a corresponding abstract transition. Therefore, proving safety in the abstract model proves safety in the concrete model.

This distinction is subtle but essential. Adding p as abstraction *vocabulary* does not claim that p is always true. Both polarities may occur in abstract states. Adding p as a concrete constraint instead removes all executions in which p is false.

3 The Soundness Failure

3.1 Constraint injection is assumption injection

The legacy path asked an LLM to identify pairs of Boolean-looking signals that appeared mutually exclusive and encoded a hint $h(X)$, typically $\neg(x \wedge y)$, as a BTOR2 **constraint**. Let the resulting model be M_h . Its reachable behaviors satisfy

$$\text{Reach}(M_h) \subseteq \text{Reach}(M).$$

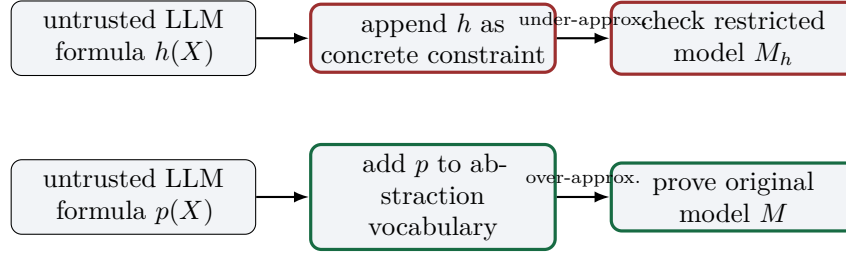


Figure 1: The old path assumed a candidate and restricted the concrete model. The repaired path uses the candidate only to refine an over-approximate abstraction; the safety verdict remains about the original model.

Consequently, M_h safe does not imply M safe. If h excludes a reachable state needed by a counterexample, the model checker can soundly prove the *restricted* model while saying nothing about the original model.

The error is a circular proof pattern: assume that a semantically suspicious state never occurs, then conclude that it never occurs. Source- or RTL-level naming intuition is especially unreliable after bit-level compilation because signals may represent encoded control, transient phases, or independent bits rather than one-hot semantic states.

3.2 Independent audit

We built an exact Z3 bit-vector encoder for the supported BTOR2 fragment and checked the invariant returned by each legacy run against the original, unmodified model using eqs. (1) to (3). Of 32 independently checkable old proofs, all 32 were rejected. The dominant audit signature was that the returned formula was inductive only in a way insufficient to establish initial truth or safety on the original model.

A separate bounded-reachability scan tested the mutex hints themselves. It found a reachable state satisfying $x \wedge y$ for 30 of 32 tested pairs. None of the 32 was established as a true invariant. The remaining two are not counted as valid: absence of a bounded counterexample is not an inductive proof.

These results invalidate the old proof count rather than merely weakening it. The correct unit of evidence is an original-model certificate or a model-checker verdict on the unchanged transition system, not UNSAT after adding an unproved assumption.

4 Sound Predicate Injection

The repaired interface is

```
pono -e ic3ia -initial-predicates predicates.json original.btor2.
```

Each line contains a typed predicate AST. Symbolic source-like names are resolved to stable internal state references, and supported Boolean and bit-vector operators are reconstructed in the IC3IA solver.

During `IC3IA::initialize()`, the implementation calls the existing `add_predicate` path. The method creates current and next abstraction labels, links them to the concrete predicate evaluations, and asks the implicit abstraction layer for a predicate-refinement constraint. It does not append the candidate to the concrete I , T , C , or B_j . Thus the candidate can be false on reachable states without creating a false safety result.

The precise soundness statement is:

Arbitrary candidate predicates may refine the abstraction vocabulary and abstract transition relation while preserving an over-approximation of the original transition system. A reported safety result is therefore a proof of the original unconstrained model, not an assertion that the candidates were themselves invariants.

This does *not* imply that bad candidates are operationally harmless. Each predicate adds an abstraction dimension and may increase SMT query size, refinement count, memory use, or runtime. Soundness and usefulness are separate properties.

The implementation also provides two independent certification paths:

- `scripts/cert_check.py` parses a returned invariant and checks C1, C2, and every C3 obligation on the original model;

Table 1: Evidence semantics used throughout the audit.

Observation	Permitted interpretation
LLM formula	Untrusted proposal only
Predicate accepted by IC3IA	Sound abstraction vocabulary; not a certified invariant
Bounded SAT witness	Refutes the candidate or establishes a bounded counterexample
Bounded UNSAT / timeout / SMT UNKNOWN	No inductive conclusion
C1, C2, and all C3 queries UNSAT	Certified inductive safety invariant
IC3IA UNSAT on original model	Valid safety result for that model/configuration

- `scripts/candidate_cert_check.py` checks a candidate conjunction directly and can apply exact Houdini elimination, removing candidates refuted by initial or one-step witnesses until a C1/C2-valid subset remains, then checking every bad property.

Unsupported forms and UNKNOWN results are explicit. No bounded-success result is silently promoted to an invariant.

5 Evaluation Protocol

The evaluation evolved into a falsification protocol designed to prevent four common confounders.

Original-model proof. Every accepted safety result is either returned by a sound model-checker run on the original BTOR2 file or independently passes exact C1/C2/C3. All bad properties are checked.

Matched expressiveness. Verifier engines and candidate generators are separate baselines. We compare LLM candidates not only with plain induction, interpolation, and IC3IA, but also with deterministic affine and quadratic template generators capable of expressing the forms attributed to the LLM.

Frozen replay and total cost. Candidate capture is separated from proof replay. Capture artifacts bind benchmark, prompt, response, candidate, and metadata hashes. Replay performs no API call. We distinguish proof-only time from

$$T_{\text{end-to-end}} = T_{\text{generation}} + T_{\text{processing}} + T_{\text{proof}}.$$

Tokens and invalid outputs remain part of the record.

Preregistered gates. Later mechanism studies freeze their population, eligibility rules, success thresholds, and stop conditions before observing the decisive outcome. A failed population prerequisite is recorded as *not run*, not converted to a smaller post-hoc experiment.

6 Experimental Results

6.1 Corrected 21-task comparison

The first sound comparison appeared to leave three affine LLM-only examples: `93.c`, `fib_37`, and `fib_05`. They were unique only relative to the tested PONO engines. A deterministic affine generator covered them once the formula language was matched.

Two nonlinear arithmetic cases, `fib_23` and `fib_30`, were then used as a stronger reliability test. Five independent captures for each case produced a Houdini-certifiable conjunction in all ten captures. This is a positive reproducibility observation: the model repeatedly found an appropriate nonlinear structure. It is not an LLM-specific expressiveness result, because a deterministic quadratic recurrence generator certifies both cases as well.

As shown in table 2, the LLM and deterministic quadratic solved sets are identical: seven candidate-certified safe tasks. Adding the ordinary engine portfolio gives both sides eight safe and two unsafe tasks. The other 11 tasks remain unresolved under the recorded configurations. Direct certification itself is cheap on the nonlinear

Table 2: Corrected full21 results. Candidate columns count directly certified safe tasks; portfolio columns add ordinary engine results.

Configuration	Certified safe	Portfolio safe	Portfolio unsafe
LLM Houdini / engine+LLM	7	8	2
Deterministic quadratic / engine+det.	7	8	2
LLM-specific after matching		0	

Table 3: Decisions from the frozen paired representation gate.

Hypothesis	Observation	Decision
Source representation adds unique solve	Source/lifted/raw solve 1/1/2 baseline-hard tasks; source unique 0	Rejected on pilot
Phase conditioning generalizes	One independent phase-only proof; threshold 3	Threshold failed
LLM routing beats structural routing	Structural router covers the three-task LLM union	Rejected
Soundness of routed proofs	12/12 routed UNSAT rows certified; 0 unsafe false-safe	Supported

successes—median certificate times are roughly 0.05–0.06 seconds—while median LLM generation is roughly 49 seconds for `fib_23` and 126 seconds for `fib_30`. Therefore a proof-only plot alone would badly misstate end-to-end cost.

6.2 Broader HWMCC residual

The broader gate structurally scanned 1,919 BTOR2 files. It identified 194 software-origin models, 89 non-array candidates, and 86 unique eligible models after content deduplication. Ordinary-engine and deterministic screens were run before freezing 11 new deterministic-hard targets for LLM capture.

Across those targets, direct LLM Houdini certified none. Raw predicate seeding helped IC3IA solve one model, `up.btor2`. That result initially looked like a useful abstraction-basis compactness observation, but it did not survive stronger deterministic controls. A 200-predicate static pool reproduced the proof, and a fixed low-complexity relational ranker reproduced it with a much smaller successful prefix. In five matched replays, 15 LLM predicates and 20 ranked predicates both solved 5/5; their median proof times were 8.115 seconds and 2.134 seconds, respectively. A ranked prefix of 15 failed while 16 succeeded, and the returned invariant at the successful boundary independently passed C1/C2/C3.

The gate therefore contributes neither a new LLM-specific solve nor a defensible compactness or runtime advantage. It does illustrate a useful distinction: a candidate set may help the abstraction even if the conjunction is not itself inductive, but the source of that basis must still be compared against deterministic alternatives.

6.3 Paired representation, phase, and routing gate

To test whether high-level representation rather than formula expressiveness was the missing advantage, we constructed a paired source/target study. The population census contained 267 officially translated tasks, of which 164 were eligible. A content- and family-independent 20-family pilot was frozen before the LLM outcomes. Each task was presented in three matched views: source C, a target-derived lifted recurrence summary, and a raw BTOR2 property-cone sketch.

The LLM selected a strict grammar route rather than emitting unrestricted proof text. Deterministic code expanded the route into global or phase-conditioned predicates, and the same original-target checker evaluated them. Sixty calls consumed 142,814 tokens and 229.16 seconds of recorded API latency; 36 routes were valid and 24 invalid.

The results in table 3 again separate correctness from utility. All 12 routed safety rows passed independent original-model certification, and no unsafe control became safe. Nevertheless, source had no unique solve, phase conditioning added only one task against a threshold of three, and the deterministic structural router covered the full three-task union reached by all LLM representation arms. Candidate-count reduction alone did not become an end-to-end advantage after API latency and invalid routes.

6.4 Reliability is not marginal value

The LLM was not simply incapable of producing useful formulas. Five-round two-tier generation solved five known linear-solvable cases in 15/15 repeated runs, and the two nonlinear cases produced certifiable candidate sets in 10/10 independent captures. These observations matter for engineering: repeated sampling can stabilize a demonstration. They do not establish marginal research value when a deterministic generator produces the same certified forms at lower cost. Reliability, coverage, uniqueness, and end-to-end utility must be reported separately.

7 Mechanism Gates and Scientific Stops

Several later ideas were promising enough to preregister but did not produce a utility experiment. Reporting these outcomes prevents development controls or near misses from being mistaken for evidence.

Modular algebraic certificates. A small kernel checks polynomial identities over $\mathbb{Z}/2^w\mathbb{Z}$ without unsound division or cancellation. It validates development certificates for `fib_23` and `fib_30`; a 20-case negative suite rejects malformed, unsupported, false-initial, and unsafe cases at the expected stages. However, the frozen 267-task natural population contains zero task eligible for the v1 contract: 39 require arrays, 221 have no supported nonlinear update SCC, and the remaining seven have nonlinear SCCs exceeding the frozen eight-branch cap. Natural utility is therefore *not run*, not failed or supported.

Inductiveness-gap decomposition. Six frozen nonlinear candidates were tested to determine whether helpers, deeper induction, or proof organization was the next bottleneck. All six fail C1 in an initial state, and exact Houdini removes every one during initial filtering. They are false conjectures, not valid invariants awaiting a better proof graph. Repairing them after inspecting the witnesses would be a new, post-hoc experiment.

Known-map transport. A transport oracle was preregistered to move certified invariants through affine state recoding, split/merge encoding, and input-latched stuttering refinement. The prerequisite census found 11 certified bases and six bases applicable to the affine transformation, below frozen requirements of 12 and eight. The gate stopped before generating transformed variants, validating a map, timing proof reuse, or calling an LLM. Transport utility remains unknown.

These stops are part of the methodology. Widening the operator language, lowering 12/8 to 11/6, or adding hand-picked variants after seeing the census would change the research question. The repository records the development capability separately from the absent confirmatory population.

8 Discussion

8.1 Where the apparent advantage went

The empirical trajectory admits a useful decomposition:

1. **Assumption effect.** The largest early gains came from removing concrete behaviors, not from better proof search.
2. **Language effect.** After sound integration, affine and quadratic formulas helped, but matched deterministic generators could instantiate the same language.
3. **Sampling effect.** More rounds improved hit rate and demo stability, not the set of formula families available to the proof.
4. **Representation/routing effect.** Source and phase information did not cross the frozen utility thresholds, and structural routing covered the LLM union.
5. **Population effect.** Two later proof mechanisms could not be evaluated on a sufficiently large independently selected population under their frozen contracts.

No single negative result establishes the final conclusion. The conclusion comes from preserving the trust boundary while progressively strengthening the non-LLM comparison and refusing to refill failed population gates post hoc.

8.2 Practical recommendations

For LLM-assisted verification systems, we recommend:

- never place an unproved LLM formula in the final model as an assumption;
- distinguish an abstraction predicate from a certified invariant;
- independently check every property on the original model and treat UNKNOWN as no result;
- compare against deterministic generators with the same syntactic and semantic expressiveness, not only against verifier-native engines;
- freeze model outputs for replay and report generation, processing, and proof time separately;
- retain invalid outputs, failures, and unsafe controls in denominators;
- preregister population adequacy and stop rules before mechanism expansion.

9 Related Work

IC3 constructs inductive clauses incrementally without conventional bounded unrolling [1]. IC3IA combines this proof organization with implicit predicate abstraction and incremental refinement for SMT theories [3]. PONO exposes these and other engines in an extensible SMT-based model-checking architecture [5]; this project modifies its initial abstraction interface rather than proposing a new IC3 calculus. BTOR2 and HWMCC provide the word-level model and benchmark setting [6, 4].

Recent work combines LLM proposals with formal verifiers in several ways. Lemur gives a sound derivation framework in which automated reasoning checks LLM-proposed lemmas [8]. Other work generates multiple loop invariants and learns to rank them to reduce verifier calls [2]. Neuro-symbolic loop-invariant inference has also combined LLMs with bounded model checking and symbolic feedback [7]. Our contribution is complementary rather than a stronger invariant-synthesis algorithm. We audit the trust boundary inside an SMT model checker and show how the empirical interpretation changes after original-model certification, matched deterministic expressiveness, frozen replay, and explicit cost accounting.

10 Threats to Validity

Scope. The conclusions apply to this PONO/IC3IA fork, the recorded HWMCC and paired software-origin populations, the supported scalar bit-vector predicate language, and the tested budgets. They do not establish that LLMs are never useful in verification, hardware design, theorem proving, or source-level program analysis.

Benchmark representativeness. The corrected 21-task set is small and enriched for arithmetic examples. The broader HWMCC gate is more systematic but still requires non-array models with preserved software-like names. The paired representation pilot contains 20 families, not a universal source-to-circuit corpus. Natural and transformed or development examples are never pooled into a coverage count.

Model and provider. The principal captures use one provider/model configuration. Stochastic reliability is measured only for selected cases. A different model may produce different candidates, but it would still need to survive the same matched and sound evaluation.

Solver and implementation. Exact checking is limited by the supported BTOR2 fragment; arrays are not fully supported by the independent certificate path. Nonlinear queries may time out. A local ASan/address-space incompatibility excluded eight potential transport-source recovery rows. The frozen study did not change binary, machine, or memory policy after observing that exclusion. These limitations are recorded as unavailable or UNKNOWN, never as proof failures or successes.

Baseline construction. The strongest low-complexity ranker for `up.btor2` was added after the initial apparent compactness observation and is labeled post hoc. It is valid falsification evidence against uniqueness, but not a preregistered estimate of how often that ranking will generalize. Conversely, deterministic quadratic “oracle” baselines test expressiveness and upper-bound marginality; they are not presented as complete invariant-synthesis algorithms.

Archival status. The scientific evidence is frozen at commit `6fdb7cfd7ddf2f50aff87a8658174bd4cfbb9b2c` and tag `soundness-audit-final-v1`. This report packages existing evidence and adds no new LLM/API call or experimental verdict. Later hypotheses marked *not run* remain unknown.

11 Conclusion

The project began with a dramatic but invalid result: LLM-generated semantic hints appeared to prove dozens of models because they were inserted as constraints. Original-model auditing showed that those were proofs of under-approximated systems. Replacing assumptions with IC3IA abstraction predicates repaired the trust boundary. The repair also exposed the true empirical effect size.

Under exact certification, matched affine and quadratic baselines, frozen replay, end-to-end accounting, and preregistered stops, every current LLM solve is covered by a deterministic portfolio and no source, phase, routing, or compactness advantage survives the relevant gate. The defensible contribution is therefore not an LLM coverage claim. It is a sound integration pattern and an evidence discipline for determining whether an apparent neuro-symbolic gain belongs to the model, the candidate language, the proof engine, or the LLM.

A Artifact and Reproduction Map

The public repository is <https://github.com/swear01/pono-llm>. The primary entry points are:

- `docs/final_claim_ledger.md`: authoritative human-readable claim boundary;
- `artifacts/final_research_summary_v1.json`: hash-bound machine-readable closure index;
- `scripts/validate_final_research_summary.py`: closure validator;
- `artifacts/phase1_2_summary_v1.json`: corrected 21-task result;
- `artifacts/gate2_summary_v1.json`: broader HWMCC gate;
- `artifacts/representation_phase_v1/`: paired representation gate;
- `scripts/cert_check.py` and `scripts/candidate_cert_check.py`: independent original-model certificate paths;
- `engines/ic3ia.cpp`: initial-predicate injection point.

The minimal closure validation command is:

```
python3 scripts/validate_final_research_summary.py
```

The PDF is built from `paper/main.tex` with BibTeX. Canonical artifacts contain relative benchmark identifiers and SHA-256 provenance. Reproducing the full matrices additionally requires the corresponding HWMCC benchmark tree, solver dependencies, and provider access only for new captures; frozen replay itself performs no API call.

B Final Claim Boundary

References

- [1] Aaron R. Bradley. SAT-based model checking without unrolling. In *Verification, Model Checking, and Abstract Interpretation*, volume 6538 of *Lecture Notes in Computer Science*, pages 70–87. Springer, 2011.
- [2] Saikat Chakraborty, Shuvendu Lahiri, Sarah Fakhoury, Akash Lal, Madanlal Musuvathi, Aseem Rastogi, Aditya Senthilnathan, Rahul Sharma, and Nikhil Swamy. Ranking LLM-generated loop invariants for program verification. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 9164–9175. Association for Computational Linguistics, 2023.

Table 4: Condensed final claim ledger.

Claim	Status	Evidence boundary
Constraint-injected hints prove original models	Rejected	32/32 checkable legacy proofs rejected
Signal-name mutex hints are bit-level invariants	Rejected	30/32 conjunctions reachable; 0 certified
Initial predicate injection preserves soundness	Supported	Original concrete model unchanged
Affine/quadratic cases are LLM-only	Rejected	Matched deterministic solved set identical
Source representation adds unique solve	Rejected on pilot	Zero source-unique task
Phase conditioning generalizes	Threshold failed	One case; threshold three
LLM routing beats structural routing	Rejected	Structural route covers LLM union
Algebraic kernel improves natural cases	Not run	Zero v1-eligible natural task
Known-map transport has utility	Not run	Population prerequisite 11/6 below 12/8
LLMs are never useful for verification	Prohibited	Outside evaluated scope

- [3] Alessandro Cimatti, Alberto Griggio, Sergio Mover, and Stefano Tonetta. IC3 modulo theories via implicit predicate abstraction. In *Tools and Algorithms for the Construction and Analysis of Systems*, volume 8413 of *Lecture Notes in Computer Science*, pages 46–61. Springer, 2014.
- [4] Hardware Model Checking Competition. HWMCC: Hardware model checking competition. <https://hwmcc.github.io/>, 2026. Accessed 2026-07-18.
- [5] Makai Mann, Ahmed Irfan, Florian Lonsing, Yahan Yang, Hongce Zhang, Kristopher Brown, Aarti Gupta, and Clark Barrett. Pono: A flexible and extensible SMT-based model checker. In Rustan Leino and Alexandra Silva, editors, *Computer Aided Verification*, volume 12760 of *Lecture Notes in Computer Science*, pages 461–474. Springer, 2021.
- [6] Aina Niemetz, Mathias Preiner, Clifford Wolf, and Armin Biere. Btor2, BtorMC and Boolector 3.0. In Hana Chockler and Georg Weissenbacher, editors, *Computer Aided Verification*, volume 10981 of *Lecture Notes in Computer Science*, pages 587–595. Springer, 2018.
- [7] Guangyuan Wu, Weining Cao, Yuan Yao, Hengfeng Wei, Taolue Chen, and Xiaoxing Ma. LLM meets bounded model checking: Neuro-symbolic loop invariant inference. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, pages 406–417. ACM, 2024.
- [8] Haoze Wu, Clark Barrett, and Nina Narodytska. Lemur: Integrating large language models in automated program verification. In *International Conference on Learning Representations*, 2024.